



BICOL UNIVERSITY COLLEGE OF SCIENCE
LEGAZPI CITY, ALBAY



IT Elect 1 – WEB DEVELOPMENT

DOCUMENTATION

(Laboratory 6, POPULATING PAGES)

SUBMITTED BY:

JERALD JAY G. BUBAN

BSIT – 3C

Part 1: Create Views using Blades templates

Create [controllers](#) that load the dashboard, feed, or equivalent pages.

AuthController

```
app > Http > Controllers > AuthController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\User;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Auth;
8  use Illuminate\Support\Facades\Hash;
9
10 5 references | 0 implementations
11 class AuthController extends Controller
12 {
13     1 reference | 0 overrides
14     public function login(): View
15     {
16         return view(view: 'auth.login');
17     }
18
19     1 reference | 0 overrides
20     function loginPost(Request $request): mixed|RedirectResponse
21     {
22         $request->validate(rules: [
23             "email" => 'required',
24             "password" => 'required',
25         ]);
26         $credentials = $request->only(keys: "email","password");
27         if (Auth::attempt(credentials: $credentials)){
28             return redirect()->route(route: 'dashboard.show', parameters: ['email' => Auth::user()->email]);
29         }
30         return redirect(to: route(name: "login"))->with(key: "error",value: "Login failed");
31     }
32
33     1 reference | 0 overrides
34     function register(): View{
35     {
36         return view(view: 'auth.register');
37     }
38
39     1 reference | 0 overrides
40     function registerPost(Request $request): RedirectResponse
41     {
42         $request->validate(rules: [
43             "fullname" => 'required',
44             "email" => 'required',
45             "password" => 'required',
46         ]);
47
48         $user = new User();
49         $user->name = $request->fullname;
50         $user->email = $request->email;
51         $user->password = Hash::make(value: $request->password);
52
53         if ($user->save()){
54             return redirect(to: route(name: "login"))->with(key: "success", value: "User created successfully");
55         }
56         return redirect(to: route(name: "register"))->with(key: "error",value: "Failed to create account");
57     }
58 }
```

- The AuthController manages user authentication by handling login and registration processes. It includes functions for displaying login and registration views, processing form submissions, validating input data, and managing user sessions.
- The login() function displays the login view, while loginPost() processes the login form submission by validating the provided email and password. It uses Laravel's Auth::attempt() method to authenticate the user, redirecting them to the dashboard on success or back to the login page with an error message on failure.
- The register() function shows the registration form, and registerPost() processes registration requests by validating the user's input. It creates a new User instance, assigning the submitted name, email, and a securely hashed password using Laravel's Hash::make() method. If the registration succeeds, the user is redirected to the login page with a success message; otherwise, they are sent back to the registration page with an error notice.
- This controller ensures secure authentication with clear user feedback based on the operation's outcome.

BooksController

```
app > Http > Controllers > BooksController.php > ...
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use Illuminate\Http\Request;
6
7 class BooksController extends Controller
8 {
9     public function loadBooks(): array
10     {
11         $posts = [
12             [
13                 'book_id' => 1,
14                 'title' => "Omniscient Reader's Viewpoint",
15                 'author' => 'Sing Shong',
16                 'rating' => 4.5,
17                 'cover' => 'asset/images/orv.jpg',
18                 'description' => "Only I know the end of this world.One day our MC finds himself stuck in the world of his favorite webnovel. Wh
19                 'num_ratings' => 121212,
20                 'num_reviews' => 20212,
21                 'genres' => ['Fantasy', 'Manga', 'Light Novel', 'Science Fiction', 'Manhwa', 'Fiction'],
22                 'comments' => [
23                     ['reader' => 'Renato Jr.',
24                      'text' => 'An incredible read! So many twists and unforgettable moments!',
25                      'time' => '1 day ago']
26                 ]
27             ],
28             [
29                 'book_id' => 2,
30                 'title' => 'Lord of the Mysteries',
31                 'author' => 'Yuan Ye',
32                 'rating' => 4,
33                 'cover' => 'asset/images/lotm.jpg',
34                 'description' => "Making up to be faced with a string of mysteries, Zhou Mingrui finds himself reincarnated as Klein Moretti in
35                 'num_ratings' => 121212
36             ]
37         ];
38     }
39
40     public function show($id): View
41     {
42         $posts = $this->loadBooks();
43
44         $book = collect(Value: $posts)->firstWhere(key: 'book_id', operator: $id);
45
46         if (!$book) {
47             abort(code: 404);
48         }
49
50         return view(view: 'book-details', data: compact(var_name: 'book'));
51     }
52 }
```

- The BooksController in this Laravel application manages book-related data and serves corresponding views. It includes two primary methods: loadBooks() and show(\$id).
- The loadBooks() method returns a collection of predefined book records, each containing details such as the book’s title, author, rating, cover image, description, genres, and reader comments. This centralized data management approach simplifies updates and ensures consistency throughout the application.
- The show(\$id) method retrieves a specific book by its book_id, searching the collection returned by loadBooks() using Laravel's collect() and firstWhere() methods. If a matching book is found, the method returns the book-details view with the corresponding data. If the book does not exist, it triggers a 404 error using the abort(404) function.
- This structure promotes organized and efficient data handling while ensuring the application responds appropriately to user requests.

DashboardController

```
app > Http > Controllers > DashboardController.php > DashboardController > index()
1  <?php
2
3  namespace App\Http\Controllers;
4  use App\Http\Controllers\BooksController;
5
6  use Illuminate\Http\Request;
7  -----
8  2 references | 0 implementations
9  class DashboardController extends Controller
10 {
11     1 reference | 0 overrides
12     public function index(): View
13     {
14         $contentController = new BooksController();
15         $posts = $contentController->loadBooks();
16
17         return view(view: 'dashboard', data: compact(var_name: 'posts'));
18     }
19 }
```

- The DashboardController manages the display of the dashboard page by loading data from the BooksController.
- The index() method, an instance of BooksController is created, and its loadBooks() method is called to retrieve book-related data. The retrieved data is stored in the \$posts variable. The method returns the dashboard view, passing the \$posts variable using Laravel's compact() function, making the data accessible within the view.
- This approach helps separate data-fetching logic from the dashboard display, promoting cleaner and more modular code.

HomeController

```
app > Http > Controllers > HomeController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4  use App\Http\Controllers\BooksController;
5  use Illuminate\Http\Request;
6  -----
7  2 references | 0 implementations
8  class HomeController extends Controller
9  {
10     1 reference | 0 overrides
11     public function index(): View
12     {
13         $contentController = new BooksController();
14         $posts = $contentController->loadBooks();
15
16         return view(view: 'home', data: compact(var_name: 'posts'));
17     }
18 }
```

- The HomeController handles the homepage display by integrating book-related data. It defines the index() method, which creates an instance of BooksController and calls its loadBooks() method to retrieve the list of books. The retrieved data is then passed to the home view using Laravel's compact() function.
- This setup ensures that the homepage dynamically displays book information managed within BooksController.

LandingPageContoller

```
app > Http > Controllers > LandingPageController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4  use App\Http\Controllers\BooksController;
5  use Illuminate\Http\Request;
6  -----
7  2 references | 0 implementations
8  class LandingPageController extends Controller
9  {
10     1 reference | 0 overrides
11     public function index(): View
12     {
13         $contentController = new BooksController();
14         $posts = $contentController->loadBooks();
15
16         return view(view: 'landing-page', data: compact(var_name: 'posts'));
17     }
18 }
```

- The LandingPageController manages the logic for displaying the application's landing page. In its index() method, it creates an instance of BooksController and invokes the loadBooks() method to fetch a collection of books. The retrieved data is then passed to the landing-page view using Laravel's compact() function, ensuring the page is dynamically populated with book-related content.

Register controllers in routes to link methods to URLs.

```
routes > web.php
1  <?php
2
3  use App\Http\Controllers\BooksController;
4  use Illuminate\Support\Facades\Route;
5  use App\Http\Controllers\HomeController;
6  use App\Http\Controllers\DashboardController;
7  use App\Http\Controllers\AuthController;
8  use App\Http\Controllers\LandingPageController;
9
10
11  Route::get(uri: '/', action: [LandingPageController::class, 'index'])->name(name: 'landing-page');
12
13
14  // Authentication Routes (only for guests)
15  Route::middleware(middleware: 'guest')->group(callback: function (): void {
16      Route::get(uri: '/login', action: [AuthController::class, 'login'])->name(name: 'login');
17      Route::post(uri: '/login', action: [AuthController::class, 'loginPost'])->name(name: 'login.post');
18      Route::get(uri: '/register', action: [AuthController::class, 'register'])->name(name: 'register');
19      Route::post(uri: '/register', action: [AuthController::class, 'registerPost'])->name(name: 'register.post');
20  });
21
22  // Authenticated Routes (for logged-in users)
23  Route::middleware(middleware: 'auth')->group(callback: function (): void {
24      Route::get(uri: '/contact', action: function (): View {
25          return view(view: 'contact');
26      });
27
28      Route::get(uri: '/home', action: [HomeController::class, 'index'])->name(name: 'home.index');
29      Route::get(uri: '/dashboard', action: [DashboardController::class, 'index'])->name(name: 'dashboard.show');
30
31
32      Route::post(uri: '/logout', action: function (): RedirectResponse {
33          auth()->logout(); // Logs out the user
34          return redirect(to: route(name: 'landing-page'));
35      })->name(name: 'logout');
36
37
38      // Content loading routes
39      Route::get(uri: '/home/content', action: [BooksController::class, 'loadBooks'])->name(name: 'home.books');
40      Route::get(uri: '/dashboard/content', action: [BooksController::class, 'loadBooks'])->name(name: 'dashboard.books');
41
42      Route::get(uri: '/books/{id}', action: [BooksController::class, 'show'])->name(name: 'books.show');
43  });
44
```

- The root URL (/) routes to the LandingPageController's index method, rendering the main landing page. Public routes intended for guests only are protected by the guest middleware. These include the login and registration pages, accessed via /login and /register, managed by AuthController. Corresponding POST routes handle form submissions for authentication and registration processes.

- Protected routes, accessible only to authenticated users, are secured by the auth middleware. These include the home and dashboard pages, managed by HomeController and DashboardController, respectively. The /contact route displays a static contact page, while the /logout route logs out users and redirects them to the landing page.
- Additionally, the application supports content management through BooksController. The loadBooks method provides content for the home and dashboard views, while individual book details can be accessed through the dynamic route /books/{id}.

Part 2: Load information such as posts, ratings, comments, pictures, etc. The information must be in the form of an array.

DASHBOARD CODE

```
resources > views > dashboard.blade.php
1  @extends(view: 'Components.Layout')
2
3  @section(section: 'styles')
4  <link rel="stylesheet" href="asset/css/dashboard.css">
5  @endsection
6
7  @section(section: 'content')
8
9  <x-navbar />
10
11 <main>
12   <section class="dashboard">
13     <div class="dashboard_container">
14       <div class="row">
15         <div class="col-md-3 rf_margin">
16           <div class="profile_container">
17             <div class="profile_pic">
18               
19             </div>
20             <div class="username">
21               <p class="username">{{ Auth::user()->name }}</p>
22             </div>
23           </div>
24         </div>
25         <div class="col-md-6 rf_margin">
26           <div class="row g-0">
27             <div class="col-md-4">
28               <div class="profile_details">
29                 <div class="rectangle">
30                   <p class="numbers">100</p>
31                   <p class="txt">Books</p>
32                 </div>
33               </div>
34             </div>
35             <div class="col-md-4">
36               <div class="profile_details">
37                 <div class="rectangle">
38                   <p class="numbers">1, 245</p>
39                   <p class="txt">Friends</p>
40                 </div>
41               </div>
42             </div>
43             <div class="col-md-4">
44               <div class="profile_details">
45                 <div class="rectangle">
46                   <p class="numbers">8</p>
47                   <p class="txt">Following</p>
48                 </div>
49               </div>
50             </div>
51           </div>
52           <div class="row left_pd">
53             <div class="col-md-12">
54               <p class="register_date">Joined in November 1, 2024</p>
55               <p class="fave_genres">Favorite Genres</p>
56               <p class="genres">Romance, Mystery/Thriller, Fantasy, Science Fiction, +5 More</p>
57             </div>
58           </div>
59           <div class="row g-0">
60             <p class="mybookshelves">My Bookshelves</p>
61             <div class="w-100"></div>
62             <div class="col-md-4 justify_right">
63               <div class="profile_details">
64                 <div class="rectangle">
65                   <p>Reviewed</p>
```

```
66         <p class="txt">(01)</p>
67     </div>
68 </div>
69 </div>
70     <div class="col-md-4 justify_left">
71         <div class="profile_details">
72             <div class="rectangle">
73                 <p>Favorites</p>
74                 <p class="txt">(01)</p>
75             </div>
76         </div>
77     </div>
78 </div>
79 </div>
80 <div class="col-md-3 rf_margin">
81     <div class="row">
82         <div class="col-md-12">
83             <div class="fave_book_container">
84                 <img src=asset/images/dashboard/storm_and_silence.png>
85             </div>
86         </div>
87     </div>
88 </div>
89 </div>
90 </div>
91 </section>
92
```

```
93 <section class="recent_reviews">
94     <div class="container">
95         <div class="d-flex justify-content-between align-items-center mb-4">
96             <h3 class="text-left">Recent Reviews</h3>
97             <button id="view-all-reviews" class="view-all-link">View All</button>
98         </div>
99         <div class="row" id="review-list">
100             @foreach (array_slice(array: $posts, offset: 0, length: 4) as $post)
101                 <div class="col-md-3 mb-4 review-item">
102                     <div class="card">
103                         <div class="reviews">
104                             <h5 class="book_title">{{ $post['title'] }}</h5>
105                             @foreach ($post['comments'] as $comment)
106                                 <span class="text">"{{ $comment['text'] }}"</span>
107                                 <span class="time">{{ $comment['time'] }}</span>
108                             @endforeach
109                         </div>
110                     </div>
111                 </div>
112             @endforeach
113
114             <!-- Hidden reviews that will be shown when the "View All" button is clicked -->
115             @foreach (array_slice(array: $posts, offset: 4) as $post)
116                 <div class="col-md-3 mb-4 review-item" style="display: none;">
117                     <div class="card">
118                         <div class="reviews">
119                             <h5 class="book_title">{{ $post['title'] }}</h5>
120                             @foreach ($post['comments'] as $comment)
121                                 <span class="text">"{{ $comment['text'] }}"</span>
122                                 <span class="time">{{ $comment['time'] }}</span>
123                             @endforeach
124                         </div>
125                     </div>
126                 </div>
127             @endforeach
128         </div>
129     </div>
130 </section>
```



```
212     } else {
213         books.forEach((book, index) => {
214             if (index >= 4) {
215                 book.classList.add('hidden');
216             }
217         });
218         this.textContent = 'View All';
219     }
220 });
221
222 document.getElementById('view-all-reviews').addEventListener('click', function () {
223     const reviewItems = document.querySelectorAll('.review-item');
224     const hiddenReviews = Array.from(reviewItems).filter(item => item.style.display === 'none');
225
226     if (hiddenReviews.length > 0) {
227         reviewItems.forEach(item => {
228             item.style.display = 'block';
229         });
230         this.textContent = 'Show Less';
231     } else {
232         reviewItems.forEach((item, index) => {
233             if (index >= 4) {
234                 item.style.display = 'none';
235             }
236         });
237         this.textContent = 'View All';
238     }
239 });
240 </script>
241 @endsection
```

RENDERED PAGES

BookBayarn!

Find the book you are looking for...

Logout

HomeDashboardContact UsBlog



Jerald Jay

100Books

1,245Friends

8Following

Joined in November 1, 2024

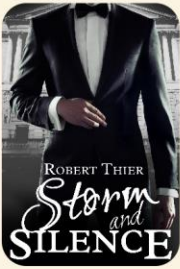
Favorite Genres

Romance, Mystery/Thriller, Fantasy, Science Fiction, +5 More

My Bookshelves

Reviewed(01)

Favorites(01)



127.0.0.1:8000/dashboard?email=jeraldjay%40gmail.com

Show Less

Recent Reviews

Omniscient Reader's Viewpoint

"An incredible read! So many twists and unforgettable moments!"

1 day ago

Lord of the Mysteries

"A thrilling mystery that keeps you on the edge of your seat. Highly recommend!"

2 days ago

A Gentle Reminder

"It's a beautiful book with gentle life lessons. Very thought-provoking."

2 days ago

Pride and Prejudice

"A classic with timeless romance and witty characters. A must-read!"

3 days ago

First Lie Wins

"A fast-paced, exciting thriller. Great book!"

5 days ago

Funny Story

"Great book! It's light and fun."

5 days ago

The Women

"A heartfelt drama. The story really stays with you after finishing it"

7 days ago

The Teacher

"I loved this mystery! It's suspenseful and keeps you guessing."

7 days ago

The Cruel Prince

"A captivating fantasy! The world-building is amazing."

2 weeks ago

Storm and Silence

"A charming historical fiction with a strong heroine. Loved every moment."

1 month ago

Waves of Memories

"A beautiful romance! It'll pull at your heartstrings."

1 month ago

The Invisible Girl

"A sweet romance with an intriguing plot. Loved it!"

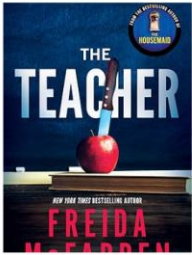
2 months ago

127.0.0.1:8000/dashboard?email=jeraldjay%40gmail.com

☆

Reviewed Books

Show Less



The Teacher

Freida McFadden

★★★★★

Renato Jr. 7 days ago



Omniscient Reader's Viewpoint

Sing Shong

★★★★☆

Renato Jr. 1 day ago

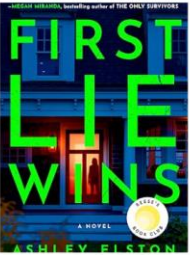


Lord of the Mysteries

Yuan Ye

★★★★☆

Renato Jr. 2 days ago



First Lie Wins

Ashley Elston

★★★★☆

Renato Jr. 5 days ago



Emily Henry

★★★★★

Renato Jr. 5 days ago



Pride and Prejudice

Jane Austen

★★★★☆

Renato Jr. 3 days ago



The Cruel Prince

Holly Black

★★★★☆

Renato Jr. 2 weeks ago

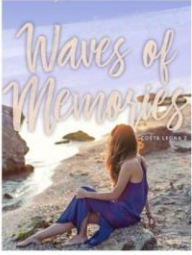


Storm and Silence

Robert Thier

★★★★☆

Renato Jr. 1 month ago



Waves of Memories

Jonaxx

★★★★☆

Renato Jr. 1 month ago



The Invisible Girl

Alexis Rose

★★★★☆

Renato Jr. 2 months ago

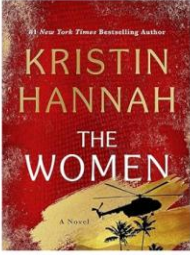


A Gentle Reminder

Bianca Sparacino

★★★★☆

Renato Jr. 2 days ago



The Women

Kristin Hannah

★★★★☆

Renato Jr. 7 days ago

BookBayarn!

Find your next favorite read!

Quick Links

Home

Categories

Community

Blog

About

Developed by 3rd Year IT students of Bicol University

bookbayarn!@gmail.com

© 2024, BookBayarn! All rights reserved.

- Doing this activity, I learned how to create views using Blade templates, such as dashboard, home, and landing-page to display different pages. I also learned how to set up routes to connect these pages with controller methods, which helped me load content and navigate between pages. I also learned how to create controllers like BooksController, HomeController, and AuthController to handle the logic for loading and managing content. To control access, we used middleware like guest and auth to make sure certain routes were only available to the right users. For example, guests could access registration and login pages, while authenticated users could visit protected routes like /home and /dashboard. We also simulated database access by loading and displaying content, such as books or posts, from arrays. This helped me better understand how data can be fetched and shown on the front end.